

Table 5-3. 8086 Memory Addressing Options Identified by the EA Abbreviations in Tables 5-4, 5-5, and 5-6

Memory Reference	Segment Register	Base Register	Index Register	Possible Displacements			Assembly Language Operand Mnemonic
				16-Bit Unsigned	8-Bit High-order Bit Extended	None	
Normal Data Memory Reference	DS (Alternate* CS, SS or ES)	None	SI	X	X	X	
			DI	X	X	X	
		BX	SI	X	X	X	
			DI	X	X	X	
	None	None	X	X	X		
		None	X				
	SS (Alternate* CS, DS or ES)	BP	SI	X	X	X	
			DI	X	X	X	
		None	None	X	X	X	
Stack	SS	SP	None				
String Data	DS	None	SI				
	ES	None	DI				
Instruction Fetch	CS	PC	None				
Branch	CS	PC	None		X		
I/O Data	DS	DX	None				
These columns contribute to OEA.							This column to be provided
These columns contribute to EA.							

Shaded rows apply to EA and DADDR.

Shaded row applies to EA and LABEL.

* The segment override allows DS or SS to be replaced by one of the other segment registers

X These are displacements that can be used to compute memory addresses.

The following abbreviations are used in Tables 5-4 and 5-5:

AH	Accumulator, high-order byte
AL	Accumulator, low-order byte
AL7	The value of register AL, high-order bit (0 or 1) extended to a byte (00 ₁₆ or FF ₁₆)
AX	Accumulator, both bytes
AX15	The value of register AX, high-order bit (0 or 1) extended to a 16-bit word (0000 ₁₆ or FFFF ₁₆)
BD	The destination is a byte operand (used only by the Assembler)
BH	B register, high-order byte
BL	B register, low-order byte
BRANCH	Program memory direct address, used in Branch addressing option shown in Tables 5-1 and 5-2
BS	The source is a byte operand (used only by the Assembler)
BX	B register, both bytes
C	Carry status
CH	C register, high-order byte
CL	C register, low-order byte
CS	Code Segment register
CX	C register, both bytes
DADDR	Data memory address operands identified in Table 5-3
DATA8	Eight bits of immediate data
DATA16	16 bits of immediate data
DH	D register, high-order byte
DI	Destination Index register
DISP	An 8-bit or 16-bit signed displacement
DISP8	An 8-bit signed displacement
DL	D register, low-order byte
DS	Data Segment register
DX	D register, both bytes
EA	Effective data memory address using any of the memory addressing options identified in Table 5-2
ES	Extra Segment register
I	Status flag set to 1
I/O	Increment/decrement selector for string operations; increment if D is 0, decrement if D is 1
LABEL	Direct data memory address, as identified in Table 5-2
N	A number between 0 and 7
O	Status flag reset to 0
OEA	Offset data memory address used to compute EA: EA = OEA + [DS] * 16
PC	Program Counter
PDX	I/O port addressed by DX register contents; port number can range from 0 through 65,536
PORT	A label identifying an I/O port number in the range 0 through 255 ₁₀
RB	Any one of the eight byte registers: AH, AL, BH, BL, CH, CL, DH, or DL
RBD	Any RB register as a destination
RBS	Any RB register as a source
RW	Any one of the eight 16-bit registers: AX, BX, CX, DX, SP, BP, SI, or DI
RWD	Any RW register as a destination
RWS	Any RW register as a source
SEGM	Label identifying a 16-bit value loaded into the CS Segment register to execute a segment jump
SFR	Status Flags register
SI	Source Index register
SP	Stack Pointer
SR	Any one of the Segment registers CS, DS, ES, or SS
SS	Stack Segment register

U Status flag modified, but undefined
V Any number in the range 0 through 255/10
X Status flag modified to reflect result
WD The destination is a word operand (used only by the Assembler)
WS The source is a word operand (used only by the Assembler)
[[]] Contents of the memory location addressed by the contents of the location enclosed in the double brackets
[] The contents of the location enclosed in the brackets
→ Data on the right-hand side of the arrow is moved to the location on the left-hand side of the arrow
↔ Contents of locations on each side of ↔ are exchanged
— The two's complement of the value under the —
≠ Not equal to

INSTRUCTION EXECUTION TIMES AND CODES

Table 5-5 lists instructions in alphabetical order, showing object codes and execution times, for the 8086 and the 8088, expressed in whole clock cycles. Execution time is the time required from beginning execution of an instruction that is in the queue to beginning execution of the next instruction in the queue. The time required to place an instruction from memory into the queue (instruction fetch time) is not shown in the table, because of queuing. Instruction fetch time occurs concurrently with instruction execution time and thus has no effect on overall timing, except as specifically noted in the table.

Instruction object codes are represented as two hexadecimal digits for instruction bytes without variations.

Instruction object codes are represented as eight binary digits for instruction bytes with variations for the instruction.

The following notation is used in Tables 5-4 and 5-5:

[] indicate an optional object code byte
a one bit choosing length:
in bit position 0 a=0 specifies 1 data byte; a=1 specifies 2 data bytes
in bit position 1 a=0 specifies 2 data bytes; a=1 specifies 1 data byte
aa two bits choosing address length:
no DISP = 00
one DISP byte = 01
two DISP bytes = 10, or 00 with bbb = 110
11 causes bbb to select a register, using the 3-bit code given below for reg.

bbb three bits choosing addressing mode:

000 EA = (BX) + (SI) + DISP
001 EA = (BX) + (DI) + DISP
010 EA = (BP) + (SI) + DISP
011 EA = (BP) + (DI) + DISP
100 EA = (SI) + DISP
101 EA = (DI) + DISP
110 EA = (BP) + DISP
111 EA = (BX) + DISP

DISP represents two hexadecimal digit memory displacement
ddd represents three binary digits identifying a destination register (see reg.)
rr two binary digits identifying a segment register:

00 = ES
01 = CS
10 = SS
11 = DS

reg three binary digits identifying a register:

16-bit 8-bit
000 = AX AL
001 = CX CL
010 = DX DL
011 = BX BL
100 = SP AH
101 = BP CH
110 = SI DH
111 = DI BH

sss represents three binary digits identifying a source register (see reg)
PPQQ represents four hexadecimal digit memory address
v one bit choosing shift length:
0 count = 1
1 count = (CL)
x "don't care" bit
YY represents two hexadecimal data digits
YYYY represents four hexadecimal data digits
z one bit where z XOR (ZF) = 1 terminates loop
. Execution time is less than or equal to instruction fetch time.
.. Includes up to eight clock cycles of overhead on each transfer due to queue maintenance. For conditional jumps, the lesser figure is when the test fails (no jump taken).

Effective Address calculation and extra clock cycles:

Extra Clock Periods		
bbb	EA	8086(1) 8088(2)
000	(BX) + (SI)	7 7
000	(BX) + (SI) + DISP8	11 11
000	(BX) + (SI) + DISP16	11 15
001	(BX) + (DI)	8 8
001	(BX) + (DI) + DISP8	12 12
001	(BX) + (DI) + DISP16	12 16
010	(BP) + (SI)	8 8
010	(BP) + (SI) + DISP8	12 12
010	(BP) + (SI) + DISP16	12 16
011	(BP) + (DI)	7 7
011	(BP) + (DI) + DISP8	11 11
011	(BP) + (DI) + DISP16	11 15
100	(SI) + (DI) or (DI)	5 5
101	or (BX)	
110	+ DISP8	9 9
111	+ DISP16	9 13
	8-bit immediate	6 6
	16-bit immediate	6 10
(1) Add another 4 clock cycles for each 16-bit operand or an odd address boundary. (2) Add another 4 clock cycles for each 16-bit operand. Substitute the clock cycles shown above wherever EA appears in Tables 5-4 and 5-5.		

Type	Mnemonic	Operands	Object Code	Clock Cycles	Statuses	Operation Performed
	MOV	RW,DADDR	8B aadbbb [DISP][DISP]	8+EA		[RW] ← [EA] Load 16 bits of data from the data memory word addressed by DADDR to register RW
	MOV	DADDR,RB	88 aasbbb [DISP][DISP]	9+EA		[EA] ← [RB] Store the data byte from register RB in the memory word addressed by DADDR
	MOV	DADDR,RW	89 aasbbb [DISP][DISP]	9+EA		[EA] ← [RW] Store the 16-bit data word from register RW in the memory word addressed by DADDR
	MOV	AL,LABEL	A0 PPOQ	10		[AL] ← [EA] Load the data memory word directly addressed by LABEL into register AL
	MOV	AX,LABEL	A1 PPOQ	10		[AX] ← [EA] Load the 16-bit data memory word directly addressed by LABEL into register AX
	MOV	LABEL,AL	A2 PPOQ	10		[EA] ← [AL] Store the 8-bit contents of register AL into the data memory word directly addressed by LABEL
	MOV	LABEL,AX	A3 PPOQ	10		[EA] ← [AX] Store the 16-bit contents of register AX into the data memory word directly addressed by LABEL
	MOV	SR,DADDR	8E a0rbbb [DISP][DISP]	8+EA		[SR] ← [EA] Load into Segment register SR the contents of the 16-bit memory word addressed by DADDR
	MOV	DADDR,SR	8C a0rbbb [DISP][DISP]	9+EA		[EA] ← [SR] Store the contents of Segment register SR in the 16-bit memory location addressed by DADDR
	XCHG	RB,DADDR	86 aarebbb [DISP][DISP]	17+EA		[RB] ← [EA] Exchange a byte of data between register RB and the data memory location addressed by DADDR
	XCHG	RW,DADDR	87 aarebbb [DISP][DISP]	17+EA		[RW] ← [EA] Exchange 16 bits of data between register RW and the data memory location addressed by DADDR
	XLAT		D7	11		[AL] ← ([AL] + [BX]) Load into AL the data byte stored in the memory location addressed by summing initial AL contents with BX contents

Table 5-4. A Summary of 8086 and 8088 Instructions (Continued)

Type	Mnemonic	Operands	Object Code	Clock Cycles	Statuses	Operation Performed
	IN	AL,PORT	E4 YY	10		[AL] ← [PORT] Load one byte of data from I/O port PORT into AL
	IN	AX,PORT	E5 YY	10		[AL] ← [PORT], [AH] ← [PORT+1] Load 16 bits of data into AX. AL receives data from I/O port PORT, AH receives data from I/O port PORT+1
	IN	AX,[DX]	ED	8		[AL] ← [PDX], [AH] ← [PDX+1] Load 16 bits of data into AX. AL receives data from the I/O port whose address is held in the DX register, AH receives data from the I/O port whose address is one higher
	OUT	AL,PORT	E6 YY	10		[PORT] ← [AL] Output one byte of data from register AL to I/O port PORT
	OUT	AX,PORT	E7 YY	10		[PORT] ← [AL], [PORT+1] ← [AH] Output 16 bits of data. The AL register contents are output to I/O port PORT, the AH register contents are output to I/O port PORT+1
	OUT	AX,[DX]	EF	8		[PORT] ← [PDX], [PORT+1] ← [PDX+1] Output 16 bits of data. The AL register contents are output to the I/O port whose address is held in the DX register, the AH register contents are output to the I/O port whose address is one higher
	LDS	RW,DADDR	C5 aasbbb [DISP][DISP]	16+EA		[RW] ← [EA], [DS] ← [EA+2] Load 16 bits of data from the memory word addressed by DADDR into register RW. Load 16 bits of data from the next sequential memory word into the DS register
	LEA	RW,DADDR	8D aasbbb [DISP][DISP]	2+EA		[RW] ← OEA Load into RW the 16-bit address displacement which, when added to the segment register contents, creates the effective data memory address
	LES	RW,DADDR	C4 aasbbb [DISP][DISP]	16+EA		[RW] ← [EA], [ES] ← [EA+2] Load 16 bits of data from the memory word addressed by DADDR into register RW. Load 16 bits of data from the next sequential memory word into the ES register
	MOV	RB,DADDR	8A aadbbb [DISP][DISP]	8+EA		[RB] ← [EA] Load one byte of data from the data memory location addressed by DADDR to register RB

Table 5-4. A Summary of 8086 and 8088 Instructions

Type	Mnemonic	Operands	Object Code	Clock Cycles	Statuses	Operation Performed
CMP	DADDR,RB		38 aassbbb [DISP][DISP]	9+EA	X	Subtract the 8-bit contents of register RB from the data memory byte addressed by DADDR. Discard the result, but adjust status flags
CMP	DADDR,RW		39 aassbbb [DISP][DISP]	9+EA	X	Subtract the 16-bit contents of register RW from the data memory word addressed by DADDR. Discard the result, but adjust status flags
DEC	DADDR		1111111a a001bbb [DISP][DISP]	15+EA	X	Decrement the contents of the memory location addressed by DADDR. Depending on the prior definition of DADDR, an 8-bit or a 16-bit memory location may be decremented
DIV	AX,DADDR		F6 aa110bbb [DISP][DISP]	(86-96)+EA	U	Divide the 16-bit contents of register AX by the 8-bit contents of the memory byte addressed by DADDR, treating both contents as signed binary numbers. Store the quotient, as a signed binary number, in AH. If the quotient is greater than FF16, execute a "divide by 0" interrupt
DIV	DX,DADDR		F7 aa110bbb [DISP][DISP]	(160-168)+EA	U	Divide the 32-bit contents of registers DX (high-order) and AX (low-order) by the 16-bit contents of the memory word addressed by DADDR. Store the quotient in AX and the remainder in DX. If the quotient is greater than FFF16, execute a "divide by 0" interrupt
IDIV	AX,DADDR		F6 aa111bbb [DISP][DISP]	(107-118)+EA	U	Divide the 16-bit contents of register AX by the 8-bit contents of the memory byte addressed by DADDR, treating both contents as signed binary numbers. Store the quotient, as a signed binary number, in AH. If the quotient is greater than 7FF16, or less than -80016, execute a "divide by 0" interrupt
IDIV	DX,DADDR		F7 aa111bbb [DISP][DISP]	(171)-190)+EA	U	Divide the 32-bit contents of register DX (high-order) and AX (low-order) by the 16-bit contents of the memory word addressed by DADDR. Treat both contents as signed binary numbers. Store the quotient in AX. Store the remainder, in AH. If the quotient is greater than 7FF16, or less than -80016, execute a "divide by 0" interrupt
MUL	AL,DADDR		F6 aa101bbb [DISP][DISP]	(86-104)+EA	X	Multiply the 8-bit contents of register AL by the contents of the memory byte addressed by DADDR. Treat both numbers as signed binary numbers. Store the 16-bit product in AX
MUL	AX,DADDR		F7 aa101bbb [DISP][DISP]	(134-160)+EA	X	Multiply the 16-bit contents of register AX by the 16-bit contents of the memory word addressed by DADDR. Treat both numbers as signed binary numbers. Store the 32-bit product in DX (high-order word) and AX (low-order word)

Table 5-4 A Summary of 8086 and 8088 Instructions (Continued)

Type	Mnemonic	Operands	Object Code	Clock Cycles	Statuses	Operation Performed
ADC	RB,DADDR		12 aadbbb [DISP][DISP]	9+EA	X	Add the contents of the 16-bit data word addressed by DADDR, plus the Carry status, to register RB
ADC	DADDR,RB		10 aassbbb [DISP][DISP]	16+EA	X	Add the 8-bit contents of register RB, plus the Carry status, to the data memory byte addressed by DADDR
ADC	DADDR,RW		11 aassbbb [DISP][DISP]	16+EA	X	Add the 16-bit contents of register RW, plus the Carry status, to the data memory word addressed by DADDR
ADD	RB,DADDR		02 aadbbb [DISP][DISP]	9+EA	X	Add the contents of the data byte addressed by DADDR to register RB
ADD	RW,DADDR		03 aadbbb [DISP][DISP]	9+EA	X	Add the contents of the 16-bit word addressed by DADDR to register RW
ADD	DADDR,RB		00 aassbbb [DISP][DISP]	16+EA	X	Add the 8-bit contents of register RB to the data memory byte addressed by DADDR
ADD	DADDR,RW		01 aassbbb [DISP][DISP]	16+EA	X	Add the 16-bit contents of register RW to the data memory word addressed by DADDR
AND	RB,DADDR		22 aadbbb [DISP][DISP]	9+EA	0	AND the 8-bit contents of register RB with the data memory byte addressed by DADDR. Store the result in RB
AND	RW,DADDR		23 aadbbb [DISP][DISP]	9+EA	0	AND the 16-bit contents of register RW with the data memory word addressed by DADDR. Store the result in RW
AND	DADDR,RB		20 aassbbb [DISP][DISP]	16+EA	0	AND the 8-bit contents of register RB with the data memory byte addressed by DADDR. Store the result in RB
AND	DADDR,RW		21 aassbbb [DISP][DISP]	16+EA	0	AND the 16-bit contents of register RW with the data memory word addressed by DADDR. Store the result in the addressed data memory word
OR	RB,DADDR		04 aadbbb [DISP][DISP]	9+EA	X	OR the 8-bit contents of register RB with the data memory byte addressed by DADDR. Store the result in RB
OR	RW,DADDR		05 aadbbb [DISP][DISP]	9+EA	X	OR the 16-bit contents of register RW with the data memory word addressed by DADDR. Store the result in RW
OR	DADDR,RB		06 aassbbb [DISP][DISP]	16+EA	X	OR the 8-bit contents of register RB with the data memory byte addressed by DADDR. Store the result in RB
OR	DADDR,RW		07 aassbbb [DISP][DISP]	16+EA	X	OR the 16-bit contents of register RW with the data memory word addressed by DADDR. Store the result in the addressed data memory word
SHL	RB,DADDR		08 aadbbb [DISP][DISP]	9+EA	X	Shift the 8-bit contents of register RB left by the data memory byte addressed by DADDR. Discard the result, but adjust status flags
SHL	RW,DADDR		09 aadbbb [DISP][DISP]	9+EA	X	Shift the 16-bit contents of register RW left by the data memory word addressed by DADDR. Discard the result, but adjust status flags
SHR	RB,DADDR		0C aadbbb [DISP][DISP]	9+EA	X	Shift the 8-bit contents of register RB right by the data memory byte addressed by DADDR. Discard the result, but adjust status flags
SHR	RW,DADDR		0D aadbbb [DISP][DISP]	9+EA	X	Shift the 16-bit contents of register RW right by the data memory word addressed by DADDR. Discard the result, but adjust status flags

Table 5-4 A Summary of 8086 and 8088 Instructions (Continued)

Secondary Memory Reference (Memory Operate) (Continued)									
Type	Mnemonic	Operands	Object Code	Clock Cycles	Statuses	Operation Performed			
	ROL	DADDR,N	110100ve aa000bbb [DISP][DISP]	N > 1 4N+20+EA	X	Rotate the contents of the data memory location addressed by DADDR left. Move the leftmost bit into the Carry status. If N = 1, then rotate one bit position. If N = CL, then register CL contents provides the number of bit positions. Depending on prior definition, DADDR may address a byte:			
	ROR	DADDR,N	110100ve aa001bbb [DISP][DISP]	N = 1 15+EA	X	As RCL, but rotate right			
	ROL	DADDR,N	110100ve aa000bbb [DISP][DISP]	N > 1 4N+20+EA	X	Rotate the contents of the data memory location addressed by DADDR left through the Carry status. If N = 1, then rotate one bit position. If N = CL, then register CL contents provide the number of bit positions. Depending on prior definition, DADDR may address a byte:			
	RCL	DADDR,N	110100ve aa011bbb [DISP][DISP]	N = 1 15+EA	X	Rotate the contents of the data memory location addressed by DADDR left through the Carry status. If N = 1, then rotate one bit position. If N = CL, then register CL contents provide the number of bit positions. Depending on prior definition, DADDR may address a byte:			
					O				
					D				
					I				
					T				
					S				
					Z				
					A				
					P				
					C				

Table 5-4. A Summary of 8086 and 8088 Instructions (Continued)

Secondary Memory Reference (Memory Operate) (Continued)									
Type	Mnemonic	Operands	Object Code	Clock Cycles	Statuses	Operation Performed			
	INC	DADDR	1111111a aa000bbb [DISP][DISP]	15+EA	X	Increment the contents of the memory location addressed by DADDR. Depending on the prior definition of DADDR, an 8-bit or a 16-bit memory location may be incremented.			
	MUL	AL,DADDR	F6 aa100bbb [DISP][DISP]	(76-83)+EA	X	Multiply the 8-bit contents of register AL by the contents of the memory location addressed by DADDR. Treat both numbers as unsigned binary numbers. Store the 16-bit product in AX.			
	MUL	F7	F7 aa100bbb [DISP][DISP]	(124-139)+EA	X	Multiply the 16-bit contents of register AX by the 16-bit contents of the memory word addressed by DADDR. Treat both numbers as unsigned binary numbers. Store the 32-bit product in DX (high-order word) and AX (low-order word).			
	NEG	DADDR	1111011a aa011bbb [DISP][DISP]	16+EA	X	Two's complement the contents of the addressed memory location. Depending on the prior definition of DADDR, an 8-bit or 16-bit memory location may be ones complemented.			
	NOT	DADDR	1111011a aa010bbb [DISP][DISP]	16+EA	X	Ones complement the contents of the addressed memory location. Depending on the prior definition of DADDR, an 8-bit or 16-bit memory location may be ones complemented.			
	OR	RB,DADDR	0A aadddbbb [DISP][DISP]	9+EA	X	OR the 8-bit contents of register RB with the data memory byte addressed by DADDR. Store the result in RB.			
	OR	RW,DADDR	0B aadddbbb [DISP][DISP]	9+EA	X	OR the 16-bit contents of register RW with the data memory word addressed by DADDR. Store the result in RW.			
	OR	DADDR,RB	08 aassbbb [DISP][DISP]	16+EA	X	OR the 8-bit contents of register RB with the data memory byte addressed by DADDR. Store the result in the data memory byte.			
	OR	DADDR,RW	09 aassbbb [DISP][DISP]	16+EA	X	OR the 16-bit contents of register RW with the data memory word addressed by DADDR. Store the result in the data memory word.			

Table 5-4. A Summary of 8086 and 8088 Instructions (Continued)

Secondary Memory Reference (Memory Operate) (Continued)										Type
Mnemonic	Operands	Object Code	Clock Cycles	Statuses						Operation Performed
SBB	DADDR,RB	18 aassbbb [DISP][DISP]	16+EA	X						Subtract the contents of 8-bit register RB from the data byte addressed by DADDR, using two's complement arithmetic. Decrement the result in data memory if the Carry status was initially set.
SBB	DADDR,RW	19 aassbbb [DISP][DISP]	16+EA	X						Subtract the contents of 16-bit register RW from the 16-bit data word addressed by DADDR, using two's complement arithmetic. Decrement the result in data memory if the Carry status was initially set.
SHL	DADDR,N	110100va aa101bbb N > 1	N = 1 15+EA; N > 1 4N+20+EA	X						This is an alternate mnemonic for SAL. As SAL, but shift right.
SHR	DADDR,N	110100va aa101bbb N > 1	N = 1 15+EA; N > 1 4N+20+EA	X						As SAL, but shift right.
SUB	RB,DADDR	2a aadddbbb [DISP][DISP]	9+EA	X						[RB] ← [RB] - [EA]
SUB	RW,DADDR	2b aadddbbb [DISP][DISP]	9+EA	X						Subtract the contents of the 16-bit data memory word addressed by DADDR from the contents of 8-bit register RW, using two's complement arithmetic.
SUB	DADDR,RB	2c aassbbb [DISP][DISP]	16+EA	X						Subtract the contents of 8-bit register RB from the data memory byte addressed by DADDR, using two's complement arithmetic.
SUB	DADDR,RW	29 aassbbb [DISP][DISP]	16+EA	X						Subtract the contents of 16-bit register RW from the 16-bit data memory word addressed by DADDR, using two's complement arithmetic.
TEST	DADDR,RB	84 aaregbbbb [DISP][DISP]	9+EA	0						AND the 8-bit contents of the data memory location addressed by DADDR with the contents of 8-bit register RB. Discard the result, but adjust status flags appropriately.

Table 5-4. A Summary of 8086 and 8088 Instructions (Continued)

Secondary Memory Reference (Memory Operate) (Continued)										Type
Mnemonic	Operands	Object Code	Clock Cycles	Statuses						Operation Performed
SAR	DADDR,N	110100va aa001bbb aa111bbb [DISP][DISP]	N = 1 15+EA; N > 1 4N+20+EA	X						As ROL, but rotate right.
SBB	RB,DADDR	1a aadddbbb [DISP][DISP]	9+EA	X						Subtract the contents of the data byte addressed by DADDR from the contents of 8-bit register RB, using two's complement arithmetic. Decrement the result in RB if the Carry status was initially set.
SBB	RW,DADDR	1b aadddbbb [DISP][DISP]	9+EA	X						Subtract the contents of the 16-bit data word addressed by DADDR from the contents of 16-bit register RW, using two's complement arithmetic. Decrement the result in RW if the Carry status was initially set.
SAL										Shift the contents of the data memory location addressed by DADDR left. Move the leftmost bit into the Carry status. If N = 1, then shift one bit position. If N = CL, then register CL contents provides the number of bit positions. Depending on prior definition, DADDR may address a byte.
										or DADDR may address a word:
										As SAL, but shift right and propagate sign:

Table 5-4. A Summary of 8086 and 8088 Instructions (Continued)

Type	Mnemonic	Operand(s)	Object Code	Clock Cycles	Statuses	Operation Performed
Jump (Cont.)	JMP	DADDR,CS	FF aa010bbb [DISP][DISP]	24+EA**		Jump indirect into a new segment. The 16-bit contents of the data memory word addressed by DADDR is loaded into PC. The next sequential 16-bit data word addressed by DADDR is loaded into PC. The next sequential 16-bit data memory word's contents is loaded into the CS segment register. [PC] → [RW]
Subroutine Call and Return	CALL	DADDR,CS	FF aa010bbb [DISP][DISP]	21+EA**		Call a subroutine in a different program segment using indirect addressing. The address of the subroutine called is stored in the 16-bit data memory word addressed by DADDR. The new CS register contents is stored in the next sequential program memory word. [SP] → [PC], [SP] → [SP-2], [PC] → [RW] Call a subroutine whose address is contained in register RW. [PC] → [DISP], [SP] → [SP] + 2 Return from a subroutine in the current segment. [PC] → [DISP], [SP] → [SP] + 2, [CS] → [DISP], [SP] → [SP] + 2 Return from a subroutine in another segment. [PC] → [DISP], [SP] → [SP] + 2 + DATA16 Return from a subroutine in the current segment and add an immediate displacement to SP. [PC] → [DISP], [SP] → [SP] + 2, [CS] → [DISP], [SP] → [SP] + 2 + DATA16 Return from a subroutine in another segment and add an immediate displacement to SP.
CALL	BRANCH	BRANCH, SEG	9A PPOQ PPOQ	19**		Call a subroutine in the current program segment using direct addressing. [SP] → [PC], [SP] → [SP] - 2, [PC] → [PC] + DISP
CALL	BRANCH	BRANCH, SEG	9A PPOQ PPOQ	28**		Call a subroutine in the current program segment using direct addressing. [SP] → [PC], [SP] → [SP] - 2, [DISP] → [PC], [SP] → [SP] - 2, [PC] → [PC] + DISP
CALL	DADDR	DADDR	FF aa010bbb [DISP][DISP]	21+EA**		Call a subroutine in the current program segment using indirect addressing. The address of the subroutine called is stored in the 16-bit data memory word addressed by DADDR. The new CS register contents is stored in the next sequential program memory word. [SP] → [PC], [SP] → [SP] - 2, [PC] → [EA] Call a subroutine in the current program segment using indirect addressing. The address of the subroutine called is stored in the 16-bit data memory word addressed by DADDR. The new CS register contents is stored in the next sequential program memory word. [SP] → [PC], [SP] → [SP] - 2, [DISP] → [PC], [SP] → [SP] - 2, [PC] → [EA], [CS] → [EA+2]
CALL	DADDR,CS	DADDR,CS	FF aa010bbb [DISP][DISP]	37+EA**		Call a subroutine in a different program segment using indirect addressing. The address of the subroutine called is stored in the 16-bit data memory word addressed by DADDR. The new CS register contents is stored in the next sequential program memory word. [SP] → [PC], [SP] → [SP] - 2, [DISP] → [PC], [SP] → [SP] - 2, [PC] → [EA], [CS] → [EA+2]
CALL	CS	CS	C8	12**		Return from a subroutine in the current segment. [PC] → [DISP], [SP] → [SP] + 2, [CS] → [DISP], [SP] → [SP] + 2
RET	RET	RET	C3	8**		Return from a subroutine in the current segment. [PC] → [DISP], [SP] → [SP] + 2
RET	RET	RET	C8	12**		Return from a subroutine in another segment. [PC] → [DISP], [SP] → [SP] + 2, [CS] → [DISP], [SP] → [SP] + 2
RET	DATA16	DATA16	C2 YYY	17**		Return from a subroutine in the current segment and add an immediate displacement to SP. [PC] → [DISP], [SP] → [SP] + 2 + DATA16
RET	CS,DATA16	CS,DATA16	CA YYY	18**		Return from a subroutine in another segment and add an immediate displacement to SP. [PC] → [DISP], [SP] → [SP] + 2, [CS] → [DISP], [SP] → [SP] + 2 + DATA16

Table 5-4. A Summary of 8086 and 8088 Instructions (Continued)

Type	Mnemonic	Operand(s)	Object Code	Clock Cycles	Statuses	Operation Performed
TEST	DADDR,RW	DADDR,RW	85 aa00 bbb [DISP][DISP]	9+EA		AND the 16-bit contents of the data memory word addressed by DADDR with the contents of 16-bit register RW. Discard the result, but adjust status flags appropriately.
XOR	RB,DADDR	RB,DADDR	32 aadbbb [DISP][DISP]	9+EA		Exclusive OR the 8-bit contents of register RB with the data memory byte addressed by DADDR. Store the result in RB.
XOR	RW,DADDR	RW,DADDR	33 aadbbb [DISP][DISP]	9+EA		Exclusive OR the 16-bit contents of register RW with the 16-bit data memory word addressed by DADDR. Store the result in RW.
XOR	DADDR,RB	DADDR,RB	30 aacbbb [DISP][DISP]	16+EA		Exclusive OR the 8-bit contents of register RB with the data memory byte addressed by DADDR. Store the result in RB.
XOR	DADDR,RW	DADDR,RW	31 aassbbb [DISP][DISP]	16+EA		Exclusive OR the 16-bit contents of register RW with the data memory word addressed by DADDR. Store the result in the addressed data memory byte.
MOV	DADDR	DADDR	C6 aa00bbb [DISP][DISP]	10+EA		Load the immediate data byte DATAB into the data memory byte addressed by DADDR.
MOV	DADDR, DATA16	DADDR, DATA16	C7 aa00bbb [DISP][DISP]	10+EA		Load the immediate 16-bit data word DATA16 into the data memory word addressed by DADDR.
MOV	RB,DATAB	RB,DATAB	10110ddd YY	4*		Load the immediate data byte DATAB into 8-bit register RB.
MOV	RW,DATA16	RW,DATA16	10111ddd YYY	4*		Load the immediate 16-bit data word DATA16 into 16-bit register RW.
JMP	BRANCH	BRANCH	111010a1 DISP [DISP]	15**		Jump direct to program memory location identified by label BRANCH. The displacement DISP which must be added to the Program Counter will be computed as an 8-bit or 16-bit signed binary number, as needed, by the assembler.
JMP	BRANCH, SEG	BRANCH, SEG	EA PPOQ PPOQ	15**		Jump direct into a new segment. BRANCH is a label which becomes a 16-bit unsigned data value which is loaded into PC. SEG is a label which becomes another 16-bit unsigned data value that is loaded into the CS segment register.
JMP	DADDR	DADDR	FF aa100bbb [DISP][DISP]	18+EA**		Jump indirect in current segment. The 16-bit contents of the data memory word addressed by DADDR is loaded into PC.

Table 5-4. A Summary of 8086 and 8088 Instructions (Continued)

Type	Mnemonic	Operands	Object Code	Clock Cycles	Statuses	Operation Performed
Immediate Operate (Continued)						
CMP	AL,DATA8	3C YY	4*	X	O D I T S Z A P C	[AL] - DATA8 adjust status flags Subtract 8-bit immediate data from AL register contents. Discard result, but
CMP	AX,DATA16	3D YYYY	4*	X	O D I T S Z A P C	[AX] - DATA16 adjust status flags Subtract 16-bit immediate data from AX register contents. Discard result, but
CMP	RB,DATA8	80 11101ddd YY	4*	X	O D I T S Z A P C	[RB] - DATA8 adjust status flags Subtract 8-bit immediate data from RB register contents. Discard result, but
CMP	RW,DATA16	100000a1 1111ddd YY YY	4*	X	O D I T S Z A P C	[RW] - DATA16 adjust status flags Subtract 16-bit immediate data from RW register contents. Discard result, but
CMP	DADDR, DATA8	80 aa11bbb [DISP][DISP] YY	10+EA	X	O D I T S Z A P C	[EA] - DATA8 Subtract 8-bit immediate data from contents of data memory byte addressed by DADDR. Discard result, but adjust status flags
CMP	DADDR, DATA16	100000a1 [DISP][DISP] YY	10+EA	X	O D I T S Z A P C	[EA] - DATA16 Subtract 16-bit immediate data from contents of 16-bit data memory word addressed by DADDR. Discard result, but adjust status flags
OR	AL,DATA8	0C YY	4*	X	O D I T S Z A P C	[AL] - [AL] OR DATA8 OR 8-bit immediate data with AL register contents
OR	AX,DATA16	0D YYYY	4*	X	O D I T S Z A P C	[AX] - [AX] OR DATA16 OR 16-bit immediate data with AX register contents
OR	RB,DATA8	80 11001ddd YY	4*	X	O D I T S Z A P C	[RB] - [RB] OR DATA8 OR 8-bit immediate data with RB register contents
OR	RW,DATA16	81 11001ddd YYYY	4*	X	O D I T S Z A P C	[RW] - [RW] OR DATA16 OR 16-bit immediate data with RW register contents
OR	DADDR, DATA8	80 aa001bbb [DISP][DISP] YY	17+EA	X	O D I T S Z A P C	[EA] - [EA] OR DATA8 OR 8-bit immediate data with contents of data memory byte addressed by DADDR
OR	DADDR, DATA16	81 aa001bbb [DISP][DISP] YY	17+EA	X	O D I T S Z A P C	[EA] - [EA] OR DATA16 OR 16-bit immediate data with contents of 16-bit data memory word addressed by DADDR
SBB	AL,DATA8	1C YY	4*	X	O D I T S Z A P C	[AL] - [AL] - DATA8 - [C] Subtract 8-bit immediate signed binary data from AL register contents using two's complement arithmetic. If the Carry status was originally 1 decrement the result
SBB	AX,DATA16	1D YYYY	4*	X	O D I T S Z A P C	[AX] - [AX] - DATA16 - [C] Subtract 16-bit immediate signed binary data from AX register contents using two's complement arithmetic. If the Carry status was originally 1 decrement the result

Table 5-4. A Summary of 8086 and 8088 Instructions (Continued)

Type	Mnemonic	Operands	Object Code	Clock Cycles	Statuses	Operation Performed
Immediate Operate						
ADD	AL,DATA8	04 YY	4*	X	O D I T S Z A P C	[AL] - [AL] + DATA8 Add 8-bit immediate data to the AL register
ADD	AX,DATA16	05 YYYY	4*	X	O D I T S Z A P C	[AX] - [AX] + DATA16 Add 16-bit immediate data to the AX register
ADD	RB,DATA8	80 11000ddd YY	4*	X	O D I T S Z A P C	[RB] - [RB] + DATA8 Add 8-bit immediate data to the RB register
ADD	RW,DATA16	81 11000ddd YYYY	4*	X	O D I T S Z A P C	[RW] - [RW] + DATA16 Add 16-bit immediate data to the RW register
ADD	DADDR, DATA8	80 aa000bbb [DISP][DISP] YY	17+EA	X	O D I T S Z A P C	[EA] - [EA] + DATA8 Add 8-bit immediate data to the data memory byte addressed by DADDR
ADD	DADDR, DATA16	81 aa000bbb [DISP][DISP] YY	17+EA	X	O D I T S Z A P C	[EA] - [EA] + DATA16 Add 16-bit immediate data to the data memory word addressed by DADDR
ADC	AL,DATA8	14 YY	4*	X	O D I T S Z A P C	[AL] - [AL] + DATA8 + [C] Add 8-bit immediate data, plus carry, to the AL register
ADC	AX,DATA16	15 YYYY	4*	X	O D I T S Z A P C	[AX] - [AX] + DATA16 + [C] Add 16-bit immediate data, plus carry, to the AX register
ADC	RB,DATA8	80 11010ddd YY	4*	X	O D I T S Z A P C	[RB] - [RB] + DATA8 + [C] Add 8-bit immediate data, plus carry, to the RB register
ADC	RW,DATA16	81 11010ddd YYYY	4*	X	O D I T S Z A P C	[RW] - [RW] + DATA16 + [C] Add 16-bit immediate data, plus carry, to the RW register
ADC	DADDR, DATA8	80 aa010bbb [DISP][DISP] YY	17+EA	X	O D I T S Z A P C	[EA] - [EA] + DATA8 + [C] Add 8-bit immediate data, plus carry, to the data memory byte addressed by DADDR
ADC	DADDR, DATA16	81 aa010bbb [DISP][DISP] YY	17+EA	X	O D I T S Z A P C	[EA] - [EA] + DATA16 + [C] Add 16-bit immediate data, plus carry, to the data memory word addressed by DADDR
AND	AL,DATA8	24 YY	4*	X	O D I T S Z A P C	[AL] - [AL] AND DATA8 AND 8-bit immediate data with AL register contents
AND	AX,DATA16	25 YYYY	4*	X	O D I T S Z A P C	[AX] - [AX] AND DATA16 AND 16-bit immediate data with AX register contents
AND	RB,DATA8	80 11100ddd YY	4*	X	O D I T S Z A P C	[RB] - [RB] AND DATA8 AND 8-bit immediate data with RB register contents
AND	RW,DATA16	81 11100ddd YYYY	4*	X	O D I T S Z A P C	[RW] - [RW] AND DATA16 AND 16-bit immediate data with RW register contents
AND	DADDR, DATA8	80 aa100bbb [DISP][DISP] YY	17+EA	X	O D I T S Z A P C	[EA] - [EA] AND DATA8 AND 8-bit immediate data with contents of data memory byte addressed by DADDR
AND	DADDR, DATA16	81 aa100bbb [DISP][DISP] YY	17+EA	X	O D I T S Z A P C	[EA] - [EA] AND DATA16 AND 16-bit immediate data with contents of 16-bit data memory word addressed by DADDR

Table 5-4. A Summary of 8086 and 8088 Instructions (Continued)

Type	Mnemonic	Operands	Object Code	Clock Cycles	Statuses	Operation Performed
Immediate Operate (Continued)						
TEST	AX,DATA16	A9 YYY	4°	0	X	AND the 16-bit immediate data and AX register contents. Discard the result
TEST	RB,DATA8	F6 11000ddd YY	5°	0	X	AND the 8-bit immediate data and RB register contents. Discard the result but adjust status flags
TEST	RW,DATA16	F7 11000ddd YYY	5°	0	X	AND the 16-bit immediate data and RW register contents. Discard the result but adjust status flags
TEST	DADDR, DATA8	F6 aa000bbb [DISP][DISP] YY	11+EA	0	X	AND the 8-bit immediate data and DADDR register contents. Discard the result but adjust status flags
TEST	DADDR, DATA16	F7 aa000bbb [DISP][DISP] YYY	11+EA	0	X	AND the 16-bit immediate data and DADDR register contents. Discard the result but adjust status flags
XOR	AL,DATA8	34 YY	4°	0	X	[AL] ← [AL] XOR DATA8
XOR	AX,DATA16	35 YYY	4°	0	X	[AX] ← [AX] XOR DATA16
XOR	RB,DATA8	80 11101ddd YY	4°	0	X	[RB] ← [RB] XOR DATA8
XOR	RW,DATA16	81 11101ddd YYY	4°	0	X	[RW] ← [RW] XOR DATA16
XOR	DADDR, DATA8	80 aa010bbb [DISP][DISP] YY	17+EA	0	X	[EA] ← [EA] XOR DATA8
XOR	DADDR, DATA16	81 aa010bbb [DISP][DISP] YYY	17+EA	0	X	[EA] ← [EA] XOR DATA16
Branch On Condition						
LOOP	DISP8	E2 DISP	5 or 17°			[CX] ← [CX] - 1 if [CX] ≠ 0 then [PC] ← [PC] + DISP8
LOOPE	DISP8	E1 DISP	6 or 18°			Decrement CX register and branch if CX contents are not 0
LOOPE	DISP8	E0 DISP	5 or 19°			Decrement CX register and branch if CX contents are not 0 and Z status is 1
LOOPNZ	DISP8	JA	4 or 16°			Decrement CX register and branch if CX contents are not 0 and Z status is 0

Table 5-4. A Summary of 8086 and 8088 Instructions (Continued)

Type	Mnemonic	Operands	Object Code	Clock Cycles	Statuses	Operation Performed
Immediate Operate (Continued)						
SBB	RB,DATA8	80 11011ddd YY	4°	X	X	Subtract 8-bit immediate signed binary data from RB register contents using two's complement arithmetic. If the Carry status was originally 1 decrement the result
SBB	DADDR, DATA8	80 aa011bbb [DISP][DISP] YY	17+EA	X	X	Subtract 8-bit immediate signed binary data from contents of data memory addressed by DADDR using two's complement arithmetic. If the Carry status was originally 1 decrement the result
SBB	DADDR, DATA16	80 aa011bbb [DISP][DISP] YY [YY]	17+EA	X	X	Subtract 16-bit immediate signed binary data from contents of 16-bit data memory addressed by DADDR using two's complement arithmetic. If the Carry status was originally 1 decrement the result
SUB	AL,DATA8	2C YY	4°	X	X	[AL] ← [AL] - DATA8
SUB	AX,DATA16	2D YYY	4°	X	X	[AX] ← [AX] - DATA16
SUB	RB,DATA8	80 11101ddd YY	4°	X	X	[RB] ← [RB] - DATA8
SUB	RW,DATA16	81 11101ddd YYY	4°	X	X	[RW] ← [RW] - DATA16
SUB	DADDR, DATA8	80 aa011bbb [DISP][DISP] YY	17+EA	X	X	[EA] ← [EA] - DATA8
SUB	DADDR, DATA16	80 aa011bbb [DISP][DISP] YY [YY]	17+EA	X	X	[EA] ← [EA] - DATA16
TEST	AL,DATA8	A8 YY	4°		X	AND the 8-bit immediate data and AL register contents. Discard the result but adjust status s

Table 5-4. A Summary of 8086 and 8088 Instructions (Continued)

Type	Mnemonic	Operand(s)	Object Code	Clock Cycles	Statuses													Operation Performed
Block Transfer and Search	MOV	Rb,Rb	8A 11 ddddss	2*														Move the contents of any Rb register to any Rb register
	MOV	RWD,RWS	8B 11 ddddss	2*														Move the contents of any Rb register to any Rb register
	MOV	SR,RW	8E 11 00ssss	2*														Move the contents of any RW register to any RW register
	MOV	RW,SR	8C 11 00dd	2*														Move the contents of any SR register to any SR register
	XCHG	AX,RW	10 01 00reg	3*														Exchange the contents of AX and any RW register
	XCHG	Rb,Rb	86 11 regreg	4*														Exchange the contents of any two Rb registers
	XCHG	RW,RW	87 11 regreg	4*														Exchange the contents of any two RW registers
	CMPS	Bd,Bs																Compare the 16-bit data words addressed by the SI and DI index registers using string data addressing
	CMPS	Wd,Ws																Compare the 16-bit data words addressed by the SI and DI index registers using string data addressing
	CMPS																	Compare the 16-bit data words addressed by the SI and DI index registers using string data addressing
Register - Register Move	MOV	AX,RW																Move a data word from the 16-bit location addressed by the SI index register to the AX register using string data addressing
	LODS	Bd,Bs																Move a data byte from the location addressed by the SI index register to the AL register using string data addressing
	LODS	Wd,Ws																Move a data word from the 16-bit location addressed by the SI index register to the AX register using string data addressing
	MOVS	Bd,Bs																Move a data byte from the location addressed by the SI index register to the extra segment location addressed by the DI register using string data addressing
	MOVS	Wd,Ws																Move a 16-bit data word from the location addressed by the SI index register to the extra segment location addressed by the DI index register using string data addressing
	MOVS																	For these instructions, the default destination segment register cannot be overridden
	CMPS	Bd,Bs																Compare the 16-bit data words addressed by the SI and DI index registers using string data addressing
	CMPS	Wd,Ws																Compare the 16-bit data words addressed by the SI and DI index registers using string data addressing
	CMPS																	Compare the 16-bit data words addressed by the SI and DI index registers using string data addressing
	CMPS																	Compare the 16-bit data words addressed by the SI and DI index registers using string data addressing
BCC (Cont.)	JPE	DISP8																See JNP
	JPF	DISP8																See JNP
	JNZ	DISP8																Branch if Z is 1
	JNS	DISP8																Branch if S is 1
	JNP	DISP8																Branch if P is 1
	JNO	DISP8																Branch if P is 1
	JNLE	DISP8																Branch if P is 1
	JNBE	DISP8																Branch if P is 1
	JNBE	DISP8																Branch if P is 1
	JNBE	DISP8																Branch if P is 1

Table B-4. A Summary of 8086 and 8088 Instructions (Continued)

Type	Mnemonic	Operand(s)	Object Code	Clock Cycles	Statuses													Operation Performed
Branch On Condition (Continued)	JAE	DISP8		4 or 16**														Branch if the CX register contents is 0
	JBE	DISP8		4 or 16**														Branch if the CX register contents is 0
	JBE	DISP8		4 or 16**														Branch if the CX register contents is 0
	JBE	DISP8		4 or 16**														Branch if the CX register contents is 0
	JBE	DISP8		4 or 16**														Branch if the CX register contents is 0
	JBE	DISP8		4 or 16**														Branch if the CX register contents is 0
	JBE	DISP8		4 or 16**														Branch if the CX register contents is 0
	JBE	DISP8		4 or 16**														Branch if the CX register contents is 0
	JBE	DISP8		4 or 16**														Branch if the CX register contents is 0
	JBE	DISP8		4 or 16**														Branch if the CX register contents is 0
Branch On Condition (Continued)	JAE	DISP8		4 or 16**														Branch if the CX register contents is 0
	JBE	DISP8		4 or 16**														Branch if the CX register contents is 0
	JBE	DISP8		4 or 16**														Branch if the CX register contents is 0
	JBE	DISP8		4 or 16**														Branch if the CX register contents is 0
	JBE	DISP8		4 or 16**														Branch if the CX register contents is 0
	JBE	DISP8		4 or 16**														Branch if the CX register contents is 0
	JBE	DISP8		4 or 16**														Branch if the CX register contents is 0
	JBE	DISP8		4 or 16**														Branch if the CX register contents is 0
	JBE	DISP8		4 or 16**														Branch if the CX register contents is 0
	JBE	DISP8		4 or 16**														Branch if the CX register contents is 0

Table B-4. A Summary of 8086 and 8088 Instructions (Continued)

Type	Mnemonic	Operands	Object Code	Clock Cycles	Statuses	Operation Performed
DIV		RBS	F0 11110ass	80-90	U	[AX] ← [AX]/[RBS] Divide the 16-bit contents of AX by the 8-bit contents of RBS. Store the integer quotient in AL and the remainder in AH. If the quotient is greater than FFFh, execute a "divide by 0" interrupt.
DIV		RWS	F7 11110ass	144-162	U	[DX] [AX] ← [DX]/[RWS] Divide the 32-bit contents of RWS by the 16-bit contents of AX and the remainder in DX. If the quotient is greater than FFFh, execute a "divide by 0" interrupt.
IDIV		RBS	F6 11111ass	101-112	U	[AX] ← [AX]/[RBS] Divide the 16-bit contents of register AX by the 8-bit contents of RBS, treating both contents as signed binary numbers. Store the quotient, as a signed binary number, in AL. Store the remainder, as an unsigned binary number, in AH. If the quotient is greater than 7Fh, or less than -80h, execute a "divide by 0" interrupt.
IDIV		RWS	F7 11111ass	165-184	U	[DX] [AX] ← [DX]/[RWS] Divide the 32-bit contents of register DX (high-order) and AX (low-order) by the 16-bit contents of RWS. Treat both contents as signed binary numbers. Store the quotient, as a signed binary number, in AX. Store the remainder, as an unsigned binary number, in AH. If the quotient is greater than 7Fh, or less than -80h, execute a "divide by 0" interrupt.
IMUL		RBS	F6 11101ass	80-98	X	[AX] ← [AL] * [RBS] Multiply the 8-bit contents of register AL by the contents of RBS. Treat both numbers as signed binary numbers. Store the 32-bit product in DX.
IMUL		RWS	F7 11101ass	128-164	X	[DX] [AX] ← [AX] * [RWS] Multiply the 16-bit contents of register AX by the 16-bit contents of RWS. Treat both numbers as signed binary numbers. Store the 32-bit product in DX.
MUL		RBS	F6 11100ass	70-77	X	[AX] ← [AL] * [RBS] Multiply the 8-bit contents of register AL by the contents of RBS. Treat both numbers as unsigned binary numbers. Store the 16-bit product in AX.
MUL		RWS	F7 11100ass	118-133	X	[DX] [AX] ← [AX] * [RWS] Multiply the 16-bit contents of register AX by the 16-bit contents of RWS. Treat both numbers as unsigned binary numbers. Store the 32-bit product in DX.
OR		RBD,RBS	0A 11dass	3 ¹	0	[RBD] ← [RBD] OR [RBS] OR the 8-bit contents of register RBS with register RBD.
OR		RWD,RWS	0B 11dass	3 ¹	0	[RWD] ← [RWD] OR [RWS] OR the 16-bit contents of register RWS with register RWD.

Table 5-4. A Summary of 8086 and 8088 Instructions (Continued)

Type	Mnemonic	Operands	Object Code	Clock Cycles	Statuses	Operation Performed
REP		N	1111001z	+2 per loop	N/D	Repeat the next sequential instruction (which must be a Block Transfer and Search instruction) until CX contents decrements to 0. Decrement CX contents on each repeat. If the next instruction is CMPPB, CMPPW, SCAB, or SCAPW then repeat until CX contents decrements to 0 or Z status does not equal N.
SCAS		BD,BS	AF	15	X/D	[AL] ← [DI], [DI] ← [DI] ± 1 Compare AL register contents with the extra segment data byte addressed by the DI index register using string data addressing.
SCAS		WD,WS	AF	15	X/D	[AX] ← [DI], [DI] ← [DI] ± 2 Compare AX register contents with the extra segment 16-bit data word addressed by the DI index register using string data addressing.
STOS		BD,BS	AA	11	X/D	[DI] ← [AL], [DI] ← [DI] ± 1 Store the AL register contents in the extra segment data memory byte addressed by the DI index register using string data addressing.
STOS		WD,WS	AB	11	X/D	[DI] ← [AX], [DI] ← [DI] ± 2 Store the AX register contents in the extra segment 16-bit data memory word addressed by the DI index register using string data addressing.
ADC		RBD,RBS	12 11dass	3 ¹	X	[RBD] ← [RBD] + [RBS] + [C] Add the 8-bit contents of register RBS, plus the Carry status, to register RBD.
ADC		RWD,RWS	13 11dass	3 ¹	X	[RWD] ← [RWD] + [RWS] + [C] Add the 16-bit contents of register RWS, plus the Carry status, to register RWD.
ADD		RBD,RBS	02 11dass	3 ¹	X	[RBD] ← [RBD] + [RBS] Add the 8-bit contents of register RBS to register RBD.
ADD		RWD,RWS	03 11dass	3 ¹	X	[RWD] ← [RWD] + [RWS] Add the 16-bit contents of register RWS to register RWD.
AND		RBD,RBS	22 11dass	3 ¹	0	[RBD] ← [RBD] AND [RBS] AND the 8-bit contents of register RBS with register RBD.
AND		RWD,RWS	23 11dass	3 ¹	0	[RWD] ← [RWD] AND [RWS] AND the 16-bit contents of register RWS with register RWD.
CBW			98	2 ¹	X	[AH] ← [AL7] Extend AL sign bit into AH.
CMP		RBD,RBS	3A 11dass	3 ¹	X	[RBD] ← [RBD] - [RBS] Subtract the contents of register RBD from register RBS. Discard the result, but adjust status flags.
CMP		RWD,RWS	3B 11dass	3 ¹	X	[RWD] ← [RWD] - [RWS] Subtract the contents of register RWD from register RWS. Discard the result, but adjust status flags.
CWD			99	5	X	[DX] ← [AX15] Extend AX sign bit into DX.

Table 5-4. A Summary of 8086 and 8088 Instructions (Continued)

Type	Mnemonic	Operands	Object Code	Clock Cycles	Operation Performed
Register Operate (Continued)					
POP	POP	RW	0011dd	8	Load the 16-bit Stack word, addressed using Stack addressing, into the 16-bit data memory word addressed by DADDR. Increment SP by 2.
POP	POP	SR	000111	8	Load the 16-bit Stack word, addressed using Stack addressing, into the 16-bit data memory word addressed by DADDR. Increment SP by 2.
POP	POP	DADDR	8F aa0000bb	17+EA	Load the 16-bit Stack word, addressed using Stack addressing, into the 16-bit data memory word addressed by DADDR. Increment SP by 2.
PUSH	PUSH	DADDR	FF aa1100bb	16+EA	Store the 16-bit contents of the data memory word addressed by DADDR in the 16-bit Stack word, addressed using Stack addressing. Decrement SP by 2.
SHR	SHR	RW,N	11010001 1101dd	N > 1 4N+8	Shift right the 8-bit contents of register RW, as illustrated for memory operate.
SHL	SHL	RW,N	11010001 1101dd	N = 1 2	Shift right the 8-bit contents of register RW, as illustrated for memory operate.
SAR	SAR	RW,N	11010001 1101dd	N > 1 4N+8	Shift right the 8-bit contents of register RW, as illustrated for memory operate.
SAR	SAR	RW,N	11010001 1101dd	N = 1 2	Shift right the 8-bit contents of register RW, as illustrated for memory operate.
SAL	SAL	RW,N	11010001 1101dd	N > 1 4N+8	Shift left the 8-bit contents of register RW, as illustrated for memory operate.
SAL	SAL	RW,N	11010001 1101dd	N = 1 2	Shift left the 8-bit contents of register RW, as illustrated for memory operate.
ROR	ROR	RW,N	11010001 1101dd	N > 1 4N+8	Rotate right the 8-bit contents of register RW, as illustrated for memory operate.
ROR	ROR	RW,N	11010001 1101dd	N = 1 2	Rotate right the 8-bit contents of register RW, as illustrated for memory operate.
ROL	ROL	RW,N	11010001 1101dd	N > 1 4N+8	Rotate left the 8-bit contents of register RW, as illustrated for memory operate.
ROL	ROL	RW,N	11010001 1101dd	N = 1 2	Rotate left the 8-bit contents of register RW, as illustrated for memory operate.
RCR	RCR	RW,N	11010001 1101dd	N > 1 4N+8	Rotate right through Carry the 8-bit contents of register RW, as illustrated for memory operate.
RCL	RCL	RW,N	11010001 1101dd	N = 1 2	Rotate right through Carry the 8-bit contents of register RW, as illustrated for memory operate.
NOT	NOT	RW	F7 1101dd	3	Ones complement the 8-bit contents of register RW.
NOT	NOT	RB	F6 1101dd	3	Ones complement the 8-bit contents of register RB.
NEG	NEG	RW	F7 1101dd	3	Two's complement the 8-bit contents of register RW.
NEG	NEG	RB	F6 1101dd	3	Two's complement the 8-bit contents of register RB.
INC	INC	RW	0100dd	2	Increment the 8-bit contents of register RW.
INC	INC	RB	FE 1100dd	3	Increment the 8-bit contents of register RB.

Table 5-4. A Summary of 8086 and 8088 Instructions (Continued)

Type	Mnemonic	Operands	Object Code	Clock Cycles	Operation Performed
Register - Register Operate (Continued)					
SBB	SBB	RWD,RWS	1A 11ddss	3	Subtract the 8-bit contents of register RWS from RWD using two's complement arithmetic. If the Carry status was originally 1, decrement the result.
SUB	SUB	RWD,RBS	2A 11ddss	3	Subtract the 8-bit contents of register RBS from RWD using two's complement arithmetic.
SUB	SUB	RWD,RWS	2B 11ddss	3	Subtract the 16-bit contents of register RWS from RWD using two's complement arithmetic.
TEST	TEST	RWD,RBS	84 11regreg	3	AND the 8-bit contents of register RBS and register RWS. Discard the result, but adjust status flags.
TEST	TEST	RWD,RWS	85 11regreg	3	AND the 16-bit contents of register RWS and register RWS. Discard the result, but adjust status flags.
XOR	XOR	RWD,RBS	30 11ddss	3	Exclusive OR the 8-bit contents of register RBS with register RWD.
XOR	XOR	RWD,RWS	31 11ddss	3	Exclusive OR the 16-bit contents of register RWS with register RWD.
AAA	AAA		37	4	ASCII adjust AL register contents for addition (as described in accompanying text).
AAD	AAD		D5 0A	60	Decimal adjust dividend in AL prior to dividing an unpacked decimal divisor.
AAM	AAM		D4 0A	83	After multiplying an unpacked decimal operand, adjust product in AX to become an unpacked decimal result. (See accompanying text for details).
AAS	AAS		3F	4	After subtracting two unpacked decimal numbers, adjust the difference in AL so that it too is an unpacked decimal number. (See accompanying text for details).
DAA	DAA		27	4	After adding two packed decimal numbers, adjust the sum in AL so that it too is a packed decimal number. (See accompanying text for details).
DAS	DAS		2F	4	After subtracting two packed decimal numbers, adjust the difference in AL so that it too is a packed decimal number. (See accompanying text for details).
DEC	DEC	RB	FE 1100dd	3	Decrement the 8-bit contents of register RB.
DEC	DEC	RW	0100dd	2	Decrement the 16-bit contents of register RW.

Table 5-4. A Summary of 8086 and 8088 Instructions (Continued)

Type	Mnemonic	Operand(s)	Object Code	Clock Cycles	Statuses	Operation Performed
ESC		DADDR	11011xxx aaxxbbbb [DISP][DISP] F4 F0	8+EA 2* 2* + 2 3+5n 3*		The contents of the data memory location addressed by DADDR is read out of memory and placed on the data bus; however, it is not input to the CPU CPU Halt Guarantee the CPU bus control during execution of the next sequential instruction The next sequential allowed memory reference instruction accesses the segment identified by Segment register SR. See Table 20-1 for allowed memory reference instructions CPU enters the WAIT state until TEST pin receives a high input signal No operation (This is the same object code as XCHG, AX, AX.)
HLT		LOCK				
SEG		SR	001reg110			
WAIT		NOP	9B 90			
Other						

Table B-4. A Summary of 8086 and 8088 Instructions (Continued)

Type	Mnemonic	Operand(s)	Object Code	Clock Cycles	Statuses	Operation Performed
	PUSH	RW	01010rr	11	O D I T S Z A P C	[SP] ← [SP] - 2, [[SP]] ← [RW or SR] Store the contents of the specified 16-bit register in the 16-bit Stack word addressed using Stack addressing. Decrement SP by 2
	PUSHF	9C	10			[SP] ← [SP] + 2, [[SP]] ← [[SPR]] Store the Status flags register contents in the 16-bit Stack word addressed using Stack addressing. Decrement SP by 2
	INT	3	CC CD YY CE	52 51 4 or 53	0 0 0 0	Execute a software interrupt and vector through table entry 3 Execute a software interrupt and vector through table entry V If the 0 status is 1, execute a software interrupt and vector through table entry - Try 1016 Return from interrupt service routine
	INRET		CF	24		
	CLC		F8	27	[C] ← 0	[C] ← 0 Clear Carry status
	CLD		FC	27	[D] ← 0	[D] ← 0 Clear Decrement/increment select
	CLI		FA	27	[I] ← 0	[I] ← 0 Clear interrupt enable status, disabling all interrupts
	CMC		F5	27	[C] ← [C]	[C] ← [C] Complement Carry status
	LAHF		9F	47		Transfer flags to AH register as follows: 7 6 5 4 3 2 1 0 Bit no. S Z O A O P I C AH register
	SAHF		9E	47		Transfer AH register contents to status flags as follows: 7 6 5 4 3 2 1 0 Bit no. S Z A P C AH register
	STC		F9	27	[C] ← 1	[C] ← 1 Set Carry status to 1
	STD		FD	27	[D] ← 1	[D] ← 1 Set Decrement/increment status to 1
	STI		FB	27	[I] ← 1	[I] ← 1 Set interrupt enable status to 1, enabling all interrupts